Name: B.Vinayani
Registration Number: 192472179

EXPERIMENT 9

Python Code Generation using Structured Prompting

AIM

To design structured prompts that instruct an LLM to generate Python programs for specified computational problems and to validate the generated code.

PROCEDURE

1. Design a structured prompt template with clear sections: Role, Task, Requirements (function name, signature, constraints), Examples and Output Format.
2. Define the computational problems (factorial, prime check, Fibonacci series, palindrome check) with their function signatures and test cases.
3. Fill the template for each problem and send it to the LLM (the program uses the OpenAI API when OPENAI_API_KEY is set; otherwise stored sample responses are used so that the validation can be demonstrated offline).
4. Extract the Python code block from the model response.
5. Validate the code in three steps: (a) syntax check using ast.parse, (b) check that the required function is defined, (c) run the function against the test cases in a separate Python process.
6. Test the validator on deliberately faulty code, display the PASS/FAIL report and capture screenshots.

CODE

Structured prompt template used:

```
### ROLE
You are a senior Python developer who writes clean, correct and efficient code.

### TASK
{task}

### REQUIREMENTS
- Define a function named `{func}` with the signature `{signature}`.
- Use only the Python standard library.
- Add a short docstring and handle edge cases.
- Do not use input() or print() inside the function.

### EXAMPLES
{examples}

### OUTPUT FORMAT
Return exactly one Python code block and no explanation.
```

exp9_pycode.py

```
"""Experiment 9 - Python code generation using structured prompting.
```

```
A structured prompt (role, task, requirements, examples, output format) is sent
to an LLM. The returned code is then validated: syntax check, function check and
execution against test cases. If OPENAI_API_KEY is not set, stored sample
responses are used so the validation pipeline can still be demonstrated.
"""
import ast
import os
import re
import subprocess
import sys
import tempfile

MODEL = "gpt-4o-mini"

PROMPT_TEMPLATE = """### ROLE
You are a senior Python developer who writes clean, correct and efficient code.

### TASK
{task}

### REQUIREMENTS
- Define a function named `{func}` with the signature `{signature}`.
- Use only the Python standard library.
- Add a short docstring and handle edge cases.
- Do not use input() or print() inside the function.

### EXAMPLES
{examples}

### OUTPUT FORMAT
Return exactly one Python code block and no explanation."""

PROBLEMS = [
    {"title": "Factorial", "func": "factorial", "signature": "factorial(n: int) -
> int",
     "task": "Compute the factorial of a non-negative integer n.",
     "tests": [((0,), 1), ((5,), 120), ((10,), 3628800)]},
    {"title": "Prime check", "func": "is_prime", "signature": "is_prime(n: int) -
> bool",
     "task": "Return True if n is a prime number, otherwise False.",
     "tests": [((1,), False), ((2,), True), ((17,), True), ((18,), False)]},
    {"title": "Fibonacci series", "func": "fibonacci", "signature": "fibonacci(n:
int) -> list",
     "task": "Return a list containing the first n Fibonacci numbers starting
from 0.",
     "tests": [((0,), []), ((1,), [0]), ((7,), [0, 1, 1, 2, 3, 5, 8])]},
    {"title": "Palindrome check", "func": "is_palindrome", "signature":
"is_palindrome(s: str) -> bool",
     "task": "Return True if the string is a palindrome, ignoring case, spaces
and punctuation.",
     "tests": [(("Madam",), True), (("A man, a plan, a canal: Panama",), True),
(("hello",), False)]},
]

# Stored responses used only when no API key is available.
OFFLINE_RESPONSES = {
    "factorial": '''```python
def factorial(n: int) -> int:
    """Return n! for a non-negative integer n."""
    if n < 0:
        raise ValueError("n must be non-negative")
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```''',
```
```

```
    "is_prime": '''```python
def is_prime(n: int) -> bool:
    """Return True if n is prime."""
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
```''',
    "fibonacci": '''```python
def fibonacci(n: int) -> list:
    """Return the first n Fibonacci numbers."""
    series = []
    a, b = 0, 1
    for _ in range(max(n, 0)):
        series.append(a)
        a, b = b, a + b
    return series
```''',
    "is_palindrome": '''```python
def is_palindrome(s: str) -> bool:
    """Check a palindrome ignoring case and non-alphanumeric characters."""
    cleaned = [ch.lower() for ch in s if ch.isalnum()]
    return cleaned == cleaned[::-1]
```''',
}

def build_prompt(problem):
    examples = "\n".join(
        f"- {problem['func']}{args if len(args) > 1 else '(' + repr(args[0]) +
')'} -> {expected!r}"
        for args, expected in problem["tests"][:2]
)
    return PROMPT_TEMPLATE.format(task=problem["task"], func=problem["func"],
                                  signature=problem["signature"],
examples=examples)

def call_llm(prompt, func):
    """Return (response_text, source). Uses OpenAI when a key is available."""
    key = os.getenv("OPENAI_API_KEY")
    if key:
        from openai import OpenAI
        client = OpenAI(api_key=key)
        reply = client.chat.completions.create(
            model=MODEL, temperature=0,
            messages=[{"role": "user", "content": prompt}])
        return reply.choices[0].message.content, f"OpenAI {MODEL}"
    return OFFLINE_RESPONSES[func], "stored sample response (offline mode)"

def extract_code(text):
    match = re.search(r"```(?:python)?\s*\n(.*?)```", text, re.DOTALL)
    return (match.group(1) if match else text).strip()

def validate_code(code, func, tests):
    """Return a list of (passed, message) results for the generated code."""
    results = []
    try:
        tree = ast.parse(code)
        results.append((True, "Syntax check"))
    except SyntaxError as err:
```
```

```
        return [(False, f"Syntax check - {err.msg} (line {err.lineno})")]

    defined = any(isinstance(node, ast.FunctionDef) and node.name == func
                  for node in ast.walk(tree))
    results.append((defined, f"Function '{func}' is defined"))
    if not defined:
        return results

    harness = (
        f"{code}\n\nTESTS = {tests!r}\npassed = 0\n"
        f"for args, expected in TESTS:\n"
        f"    if {func}(*args) == expected:\n        passed += 1\n"
        f"print(passed, len(TESTS))\n"
    )
    with tempfile.NamedTemporaryFile("w", suffix=".py", delete=False) as handle:
        handle.write(harness)
        path = handle.name
    try:
        run = subprocess.run([sys.executable, "-I", path], capture_output=True,
                             text=True, timeout=5)
        if run.returncode != 0:
            last = run.stderr.strip().splitlines()[-1] if run.stderr.strip() else
"error"
            results.append((False, f"Execution failed - {last}"))
        else:
            passed, total = map(int, run.stdout.split())
            results.append((passed == total, f"{passed}/{total} test cases
passed"))
    except subprocess.TimeoutExpired:
        results.append((False, "Execution timed out"))
    finally:
        os.remove(path)
    return results


def show(results):
    for ok, message in results:
        print(f"  [{'PASS' if ok else 'FAIL'}] {message}")


def main():
    summary = []
    for number, problem in enumerate(PROBLEMS, start=1):
        prompt = build_prompt(problem)
        if number == 1:
            print("=== Structured prompt sent to the LLM (Problem 1) ===")
            print(prompt)
        response, source = call_llm(prompt, problem["func"])
        code = extract_code(response)
        print(f"\n=== Problem {number}: {problem['title']}  |  source: {source}
===")
        print(code)
        results = validate_code(code, problem["func"], problem["tests"])
        show(results)
        summary.append((problem["title"], all(ok for ok, _ in results)))

    print("\n=== Validator self-test with faulty code ===")
    print("Case A: missing colon (syntax error)")
    show(validate_code("def factorial(n)\n    return n", "factorial", [((3,),
6)]))
    print("Case B: wrong logic (adds instead of multiplies)")
    show(validate_code("def factorial(n):\n    return sum(range(1, n + 1))",
"factorial",
                       [((3,), 6), ((5,), 120)]))

    print("\n=== Summary ===")
    for title, ok in summary:
```

```
        print(f"  {title:<18} {'PASS' if ok else 'FAIL'}")


if __name__ == "__main__":
    main()
```

OUTPUT

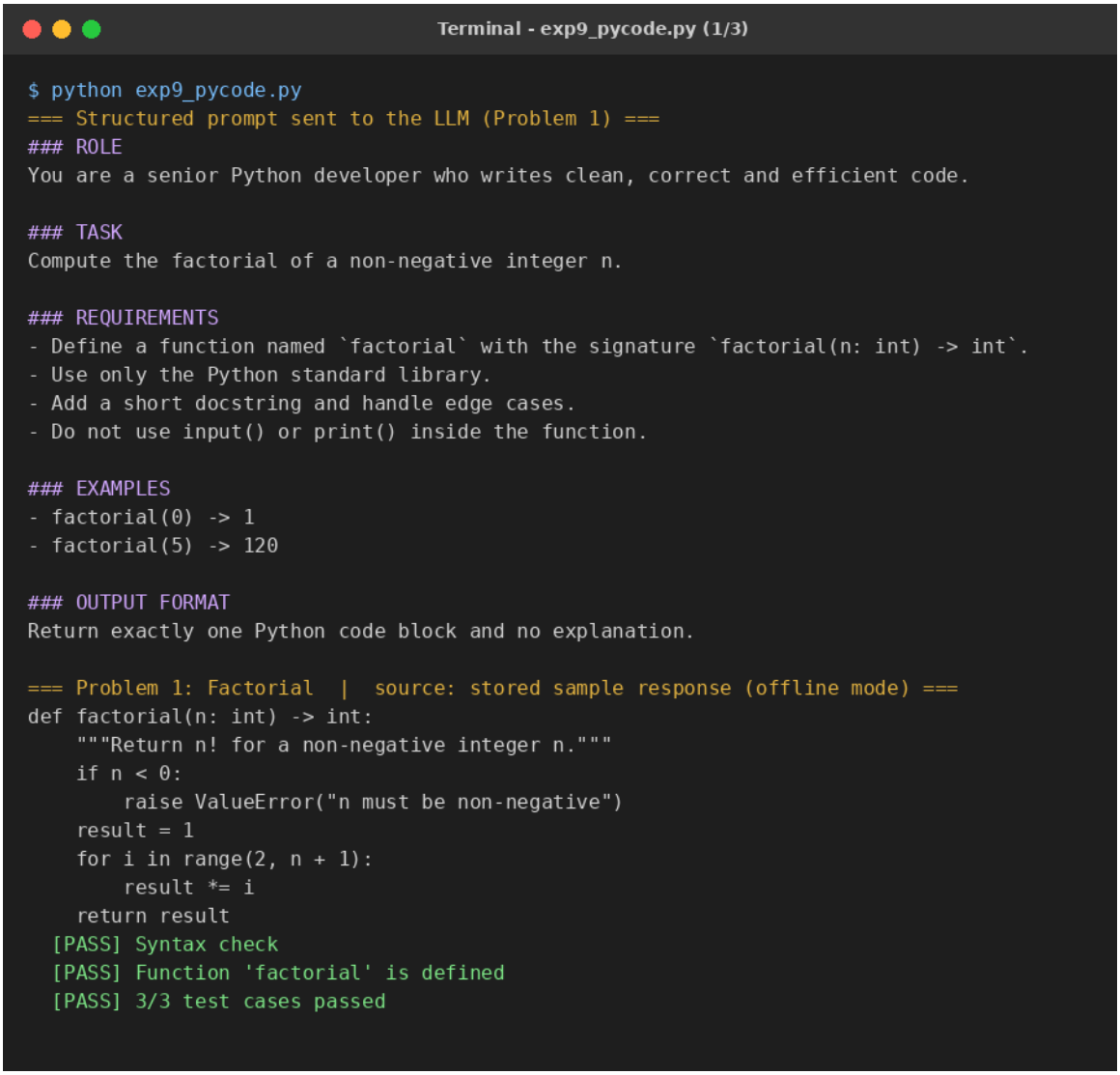


Figure 1: Sample output of the program (part 1 of 3)

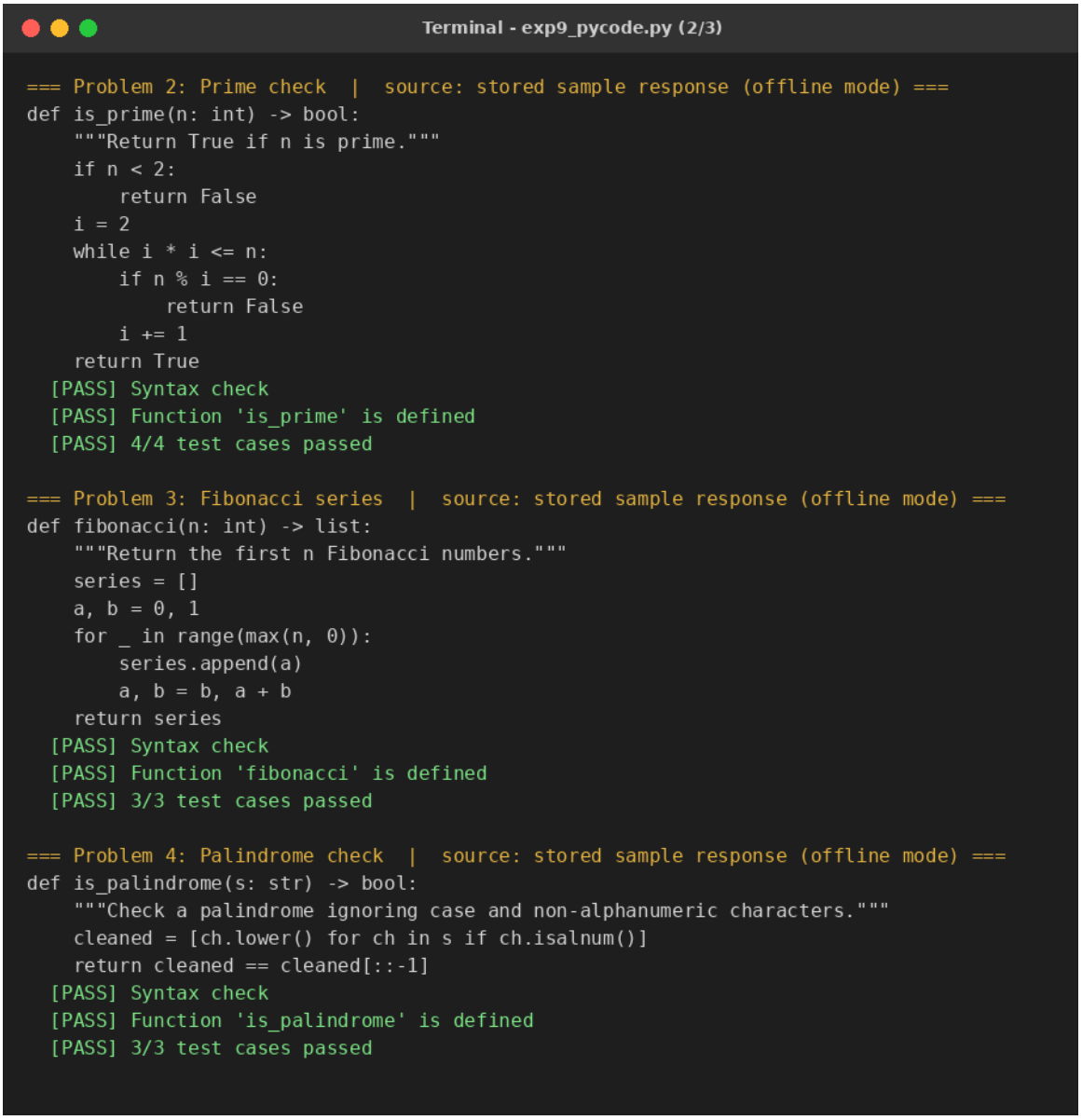


Figure 2: Sample output of the program (part 2 of 3)

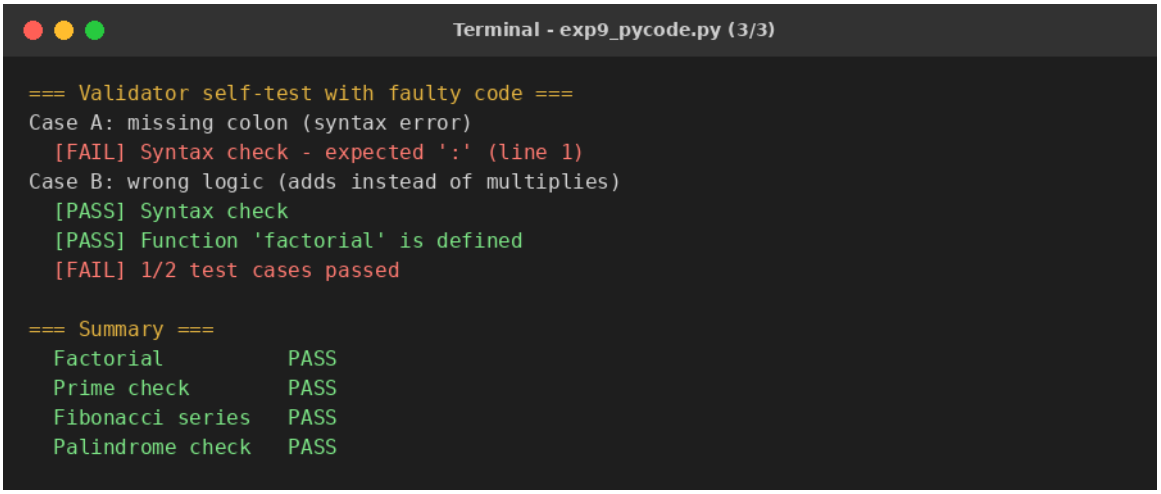


Figure 3: Sample output of the program (part 3 of 3)

RESULT

The structured prompts produced correct Python functions for all four problems. Every generated program passed the syntax check, the function-definition check and all of its test cases (3/3, 4/4, 3/3 and 3/3). The validator also correctly rejected the two faulty programs: a syntax error was detected by ast.parse and wrong logic was detected by failing test cases. In the run shown, stored sample responses were used in place of a live API call.

CONCLUSION

Structured prompting (role, task, requirements, examples and output format) gives clear, repeatable instructions to an LLM and produces code in a predictable format. Generated code should never be trusted blindly; automatic validation with syntax checks and test cases confirms correctness.